

Laboratorio 3 — MQTT hacia Azure IoT Central

Protocolo visible, mediciones y comparación con el SDK

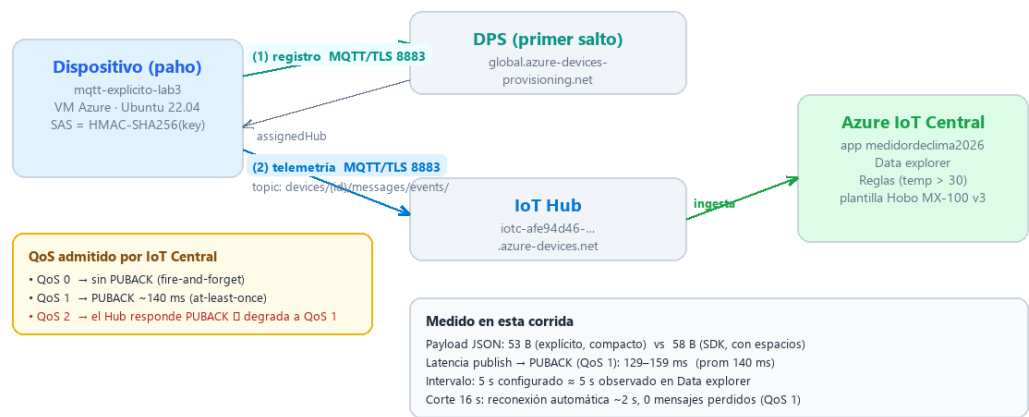
IoT + Cloud + Sistemas Distribuidos · UNAB · Escenario UNAB-Ambiental · App IoT Central: medidordeclima2026

1. El protocolo real: MQTT/TLS → DPS → IoT Hub

Los dispositivos del Lab 2 no hablan un protocolo propietario: usan MQTT sobre TLS (puerto 8883). El primer salto es el Device Provisioning Service (DPS), que registra al dispositivo y le devuelve el IoT Hub asignado; a partir de ahí la telemetría se publica al topic devices/{deviceId}/messages/events/ autenticando con un token SAS (HMAC-SHA256 de la clave del dispositivo). Se verificó abriendo el Data explorer de IoT Central: el JSON recibido coincide con la plantilla Hobo MX-100 v3 (temperature, humedad, illuminance).

Flujo MQTT → DPS → IoT Hub → Azure IoT Central

Laboratorio 3 · UNAB-Ambiental · cliente MQTT explícito (paho) en la VM de Azure



2. Camino B — cliente MQTT explícito (paho-mqtt) en la VM

En la misma VM de Azure corre un segundo dispositivo (mqtt-explicito-lab3) que NO usa azure-iot-device: genera el SAS a mano, hace el registro DPS por MQTT, se conecta al IoT Hub por MQTT/TLS y publica las 3 variables. El publish 'a mano' aparece en la misma app del Lab 1. El SDK (2c5qb72v4u) y el ESP32 de Wokwi (ve8oksIkz9) siguen conectados en paralelo sobre la misma plantilla.

```
MQTT explícito (paho): DPS por MQTT + SAS + publish QoS1
00:59:47 [SAS] Token de DPS generado (sr=0ueR0FCA1E/registrations/mqtt-explicito-lab3, expira en 3600s)
00:59:47 [DPS] Conectando por MQTT/TLS a global.azure-devices-provisioning.net:8883 ...
00:59:47 [DPS] CONBACK rc=0 -> suscribiendo a $dps/registrations/messages
00:59:47 [DPS] PUBLISH registro -> $dps/registrations/PUT/iotdps-register/ ({"registrationId": "mqtt-explicito-lab3"})
00:59:47 [DPS] 202 Assigning... operationId recibido, consultando estado
00:59:49 [DPS] 200 assigned -> hub=iotc-afe94d46-6329-4fa6-bcce-2b3d5b5aac5b.azure-devices.net status=assigned
00:59:49 [SAS] Token de IoT Hub generado (sr=iotc-afe94d46-6329-4fa6-bcce-2b3d5b5aac5b.azure-devices.net/devices/mqt...
00:59:49 [HUB] Conectando por MQTT/TLS a iotc-afe94d46-6329-4fa6-bcce-2b3d5b5aac5b.azure-devices.net:8883 (QoS=1)
00:59:49 [HUB] CONBACK rc=0 (conectado)
00:59:49 [HUB] Suscrito a métodos directos: $iothub/methods/POST/#
00:59:49 [TX #1] topic=devices/mqtt-explicito-lab3/messages/events/ qos=1 bytes=54 payload={"temperature":23.5,"humid...
00:59:49 [ACK #1] PUBACK en 144 ms (qos=1)
00:59:51 [TX #2] topic=devices/mqtt-explicito-lab3/messages/events/ qos=1 bytes=54 payload={"temperature":31.2,"humid...
00:59:51 [ACK #2] PUBACK en 159 ms (qos=1)
00:59:53 [TX #3] topic=devices/mqtt-explicito-lab3/messages/events/ qos=1 bytes=54 payload={"temperature":23.8,"humid...
00:59:53 [ACK #3] PUBACK en 141 ms (qos=1)
00:59:55 [TX #4] topic=devices/mqtt-explicito-lab3/messages/events/ qos=1 bytes=54 payload={"temperature":22.0,"humid...
00:59:55 [ACK #4] PUBACK en 132 ms (qos=1)
00:59:57 [TX #5] topic=devices/mqtt-explicito-lab3/messages/events/ qos=1 bytes=54 payload={"temperature":21.6,"humid...
00:59:57 [ACK #5] PUBACK en 143 ms (qos=1)
00:59:59 [TX #6] topic=devices/mqtt-explicito-lab3/messages/events/ qos=1 bytes=54 payload={"temperature":23.5,"humid...
00:59:59 [ACK #6] PUBACK en 152 ms (qos=1)
01:00:01 [TX #7] topic=devices/mqtt-explicito-lab3/messages/events/ qos=1 bytes=54 payload={"temperature":22.8,"humid...
01:00:01 [ACK #7] PUBACK en 141 ms (qos=1)
01:00:03 [TX #8] topic=devices/mqtt-explicito-lab3/messages/events/ qos=1 bytes=54 payload={"temperature":24.0,"humid...
01:00:03 [ACK #8] PUBACK en 131 ms (qos=1)
01:00:05 [TX #9] topic=devices/mqtt-explicito-lab3/messages/events/ qos=1 bytes=54 payload={"temperature":25.7,"humid...
01:00:05 [ACK #9] PUBACK en 129 ms (qos=1)
01:00:07 [TX #10] topic=devices/mqtt-explicito-lab3/messages/events/ qos=1 bytes=54 payload={"temperature":25.3,"humid...
01:00:07 [ACK #10] PUBACK en 144 ms (qos=1)
01:00:09 [HUB] DESCONECTADO (rc=0). paho intentará reconectar con backoff...
01:00:09 [HUB] RESUMEN -----
01:00:09 [HUB] mensajes:10 payload_bytes: min=54 max=54 prom=54.0 QoS=1
01:00:09 [HUB] latencia PUBACK ms: min=129 max=152 prom=140 (n=10)
01:00:09 [HUB] CSV escrito en /home/azureuser/laboratorio3/mediciones/qos1.csv
```

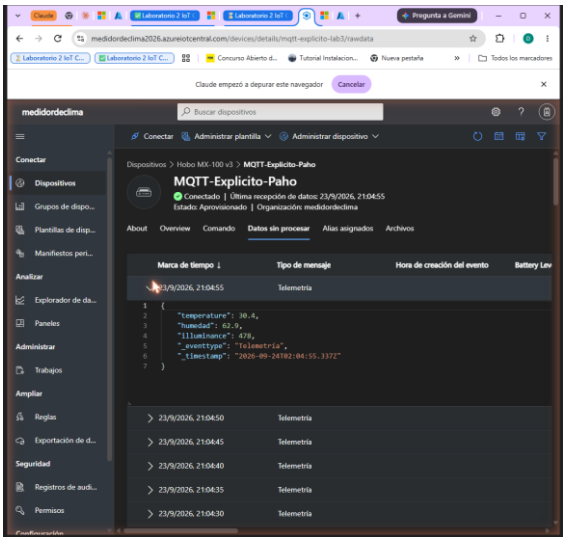


Fig. 1 — (izq.) log real: DPS por MQTT + SAS + publish QoS 1 con PUBACK. (der.) el mismo publish visible en IoT Central: device Conectado y JSON con las 3 variables mapeadas.

3. Mediciones sobre IoT Central

Métrica	SDK (azure-iot-device)	MQTT explícito (paho)	ESP32 Wokwi (Arduino)
Payload JSON	58 B (con espacios)	53 B (compacto)	~55 B

Intervalo config. → observado	5 s → ~5 s	5 s → ~5 s	~5 s
Latencia publish→PUBACK	oculta por el SDK	129–159 ms (prom 140)	N/A (QoS 0)
Latencia publish→visible	pocos s (1–5 s)	pocos s (1–5 s)	pocos s
Corte de red 10–20 s	reconecta solo	reconecta ~2 s, encola y hace flush (0 pérdida)	reconecta en loop()
QoS usado	1 (fijo)	0 y 1 (a elección)	0 (límite PubSubClient)
QoS 2	n/a	Hub responde PUBACK ⇒ degrada a QoS 1	no soportado

4. Comparación de los tres caminos

Criterio	SDK Python	MQTT explícito	ESP32 Arduino
Facilidad de implementación	Alta (pocas líneas)	Media (SAS/topics/QoS a mano)	Media-baja (C++, mbedTLS)
Control del protocolo	Bajo (caja negra)	Total	Alto (PubSubClient)
Payload / formato	No se controla	Se controla (compacto)	Se controla en firmware
Reconexión	Automática	Auto (loop paho); renovar SAS	Manual en loop()
Útil para...	Producción rápida	Depurar / integrar / portar	Hardware real / edge

5. Conclusión — QoS y cuándo usar cada camino

QoS: IoT Central (IoT Hub) admite QoS 0 y 1. QoS 2 no está implementado: al publicar en QoS 2 el Hub responde con PUBACK (el acuse de QoS 1) en lugar del handshake PUBREC/PUBREL/PUBCOMP, es decir, lo degrada a QoS 1 y no ofrece la garantía exactly-once. Para telemetría ambiental QoS 1 es el punto óptimo (entrega al-menos-una-vez con encolado y reconexión, como demostró la prueba de corte); QoS 0 sirve para datos de alta frecuencia donde perder alguna muestra es tolerable.

Cuándo cada uno: el SDK conviene para desarrollar rápido sin preocuparse por el protocolo (producción típica). El cliente MQTT explícito conviene cuando se necesita control fino —formato del payload, elegir QoS, integrarlo en un cliente propio, depurar el tráfico o portarlo a un lenguaje sin SDK—. El ESP32/Arduino es el camino natural en hardware real: su firmware ya es MQTT explícito.